

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To understand the working principle of convolutional neural networks by implementing convolution, pooling, feature-map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to:

1. explain the convolution operation;
2. compute output dimensions using stride and padding;
3. visualize feature maps;
4. implement max pooling and average pooling;
5. calculate CNN parameters;
6. build a CNN for image classification; and
7. evaluate CNN performance.

3. Background Theory

A convolutional neural network consists of convolution, activation, pooling, and fully connected layers. The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n),$$

where X is the input image, K is the kernel (filter), and Y is the feature map. The output feature-map size is

$$\left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1,$$

where N is the input size, F is the filter size, P is the padding, and S is the stride.

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract visual features such as edges, textures, corners, and smooth regions. At each position, corresponding image and kernel values are multiplied elementwise and summed to produce one value in the output feature map.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

It is useful as a baseline and for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction:

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}.$$

Applications include object detection, lane detection, and face recognition.

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction:

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

Applications include text detection, boundary extraction, and medical image analysis.

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}.$$

Applications include edge enhancement, satellite image analysis, and medical imaging.

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}.$$

Applications include image enhancement, optical character recognition, and face recognition.

6. Box Blur Kernel

The box blur replaces each pixel with the average of its neighboring pixels:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

Applications include noise removal, image smoothing, and image preprocessing.

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving its overall structure better than a simple average filter:

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}.$$

Applications include noise reduction, computer-vision preprocessing, and feature extraction.

8. Emboss Kernel

The emboss kernel creates a three-dimensional appearance by emphasizing directional intensity changes:

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}.$$

Applications include texture analysis and artistic image effects.

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}.$$

Applications include image segmentation, boundary detection, and shape extraction.

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction:

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

Applications include motion simulation, image restoration, and video processing.

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve the image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box blur	3×3	Smooth the image	Noise removal
Gaussian blur	3×3	Reduce noise	Computer vision
Emboss	3×3	Create a 3D effect	Artistic filtering
Outline	3×3	Extract boundaries	Segmentation
Motion blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the input image and kernel

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

The four convolution positions give

$$\begin{aligned} 1(1) + 2(0) + 4(0) + 5(1) &= 6, & 2(1) + 3(0) + 5(0) + 6(1) &= 8, \\ 4(1) + 5(0) + 7(0) + 8(1) &= 12, & 5(1) + 6(0) + 8(0) + 9(1) &= 14. \end{aligned}$$

Therefore, the feature map is

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}.$$

5. Numerical Example 2: Max Pooling

For the feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix},$$

a 2×2 max-pooling filter with stride 2 produces maxima 8, 3, 6, and 9. The output is

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}.$$

6. Numerical Example 3: Parameter Calculation

Suppose the input image has size $32 \times 32 \times 3$, and a convolution layer has 16 filters with 3×3 kernels. Including one bias per filter, the trainable parameter count is

$$(3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448.$$

7. Dataset

Use the CIFAR-10 dataset:

- training images: 50,000;
- testing images: 10,000;
- classes: 10; and
- image size: $32 \times 32 \times 3$.

8. Experimental Procedure

Task 1: Load CIFAR-10

- Display ten sample images.
- Print the dataset dimensions.
- Plot the class distribution.

Task 2: Implement a Convolution Layer

Compare 3×3 , 5×5 , and 7×7 kernels, and record the resulting feature-map sizes.

Task 3: Study Hyperparameters

Compare strides 1 and 2, and compare `same` and `valid` padding. Compute the output dimensions.

Task 4: Visualize Feature Maps

Visualize at least eight feature maps after the first convolution layer.

Task 5: Compare Pooling Methods

Compare max pooling and average pooling in terms of output size and accuracy.

Task 6: Construct and Train the CNN

Use the architecture

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool
 $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool
 $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax.

Train with the Adam optimizer for 20 epochs using a batch size of 32.

Task 7: Evaluate the Model

Report accuracy, precision, recall, F1-score, the confusion matrix, and the classification report.

9. Mandatory Plots

Generate the following plots and write a two- or three-line inference for each:

1. sample images;
2. class distribution;

3. training accuracy;
4. validation accuracy;
5. training loss;
6. validation loss;
7. feature maps; and
8. confusion matrix.

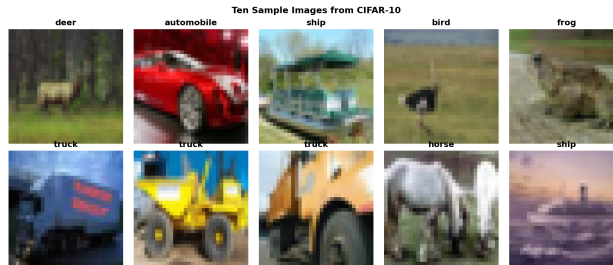


Figure 1: Sample training images

the above image consists of 10 sample images from the the dataset which are 32x32 pixelated image

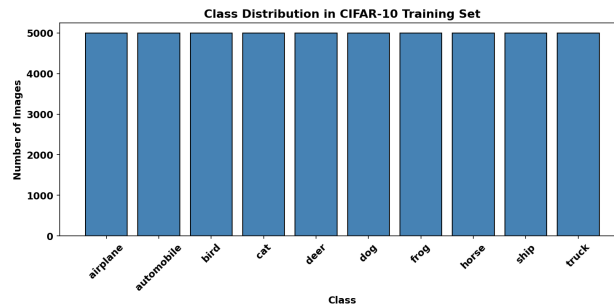


Figure 2: class distribution

from the above image of the class distribution we can see that there are 10 classes with 5000 images for each class, this shows that the following dataset is balanced for each class

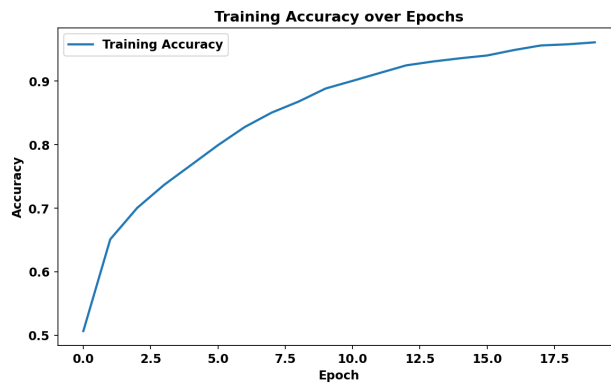


Figure 3: training accuracy

from the above graph, we can first see that the initial epochs face a steep increase in the initial training accuracy and then it starts to steadily increase after 6 epochs and the lack of a plateau line in the line graph shows that the model had a sufficient capacity

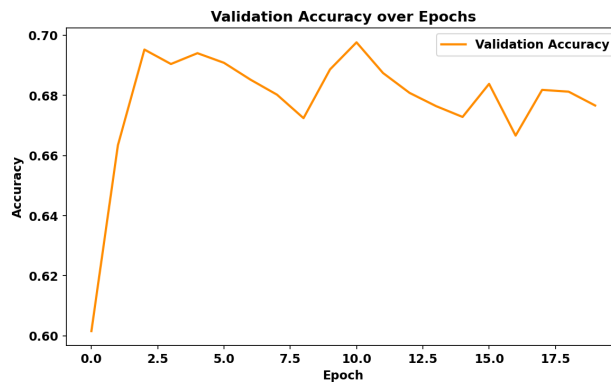


Figure 4: validation accuracy

unlike the training accuracy vs epoch graphs, the validation accuracy is showing different results where we can see steady and steep increase in the initial epoch to the accuracy which shows that the model has learnt the generalization of the patterns and then the consequent values and the increase and decrease is due to overfitting



Figure 5: training loss

the steady decrease in the training loss shows the model was able to reach convergence as the values didn't oscillate, we can see the steady but slow decrease in the loss and then it quickly decreased

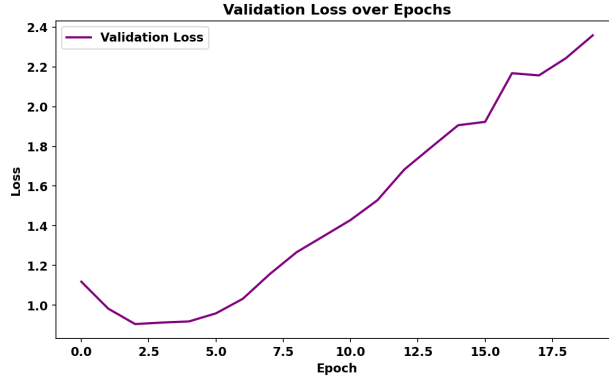


Figure 6: validation loss

unlike the training loss graph where it starts from a huge loss value, the validation loss is initially very small and it reaches the minimum in the epoch around 3 and it started to ascent after this. while parallelly reviewing with the validation accuracy vs epochs graph, we can infer that the increase the validation loss is due to the overfitting of the model

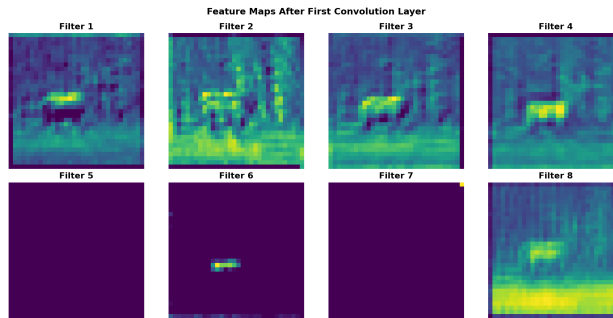


Figure 7: feature maps

from the sample feature maps presented above, we can see that model was able to learn the boundaries of the image and the contrast and the overall structure of the image. the 5th to 7th feature maps of the images indicate that the filters became dead or underutilized during the training time

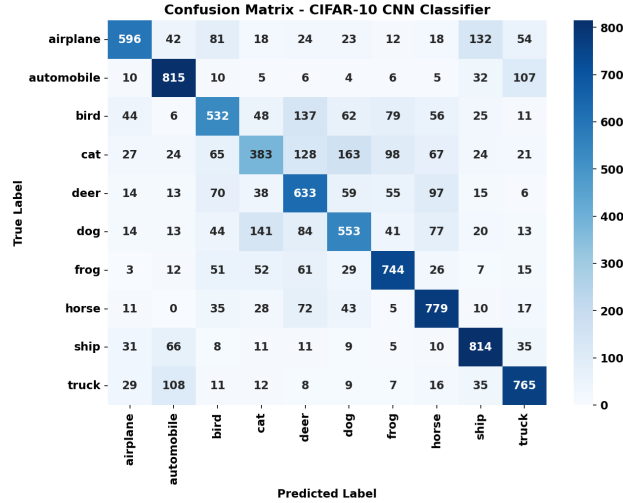


Figure 8: confusion matrix

the following confusion matrix shows majority of the images correctly classified as the dominant numbers are presented diagonally. we can see that vehicles are mostly correctly classified and animals are mostly having higher numbers of misclassification, this is because that model is not able to classify only with the structure as these specific animals do look similar to each other

10. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

$$\text{Output size} = (64 - 5 + 2 \times 2) / 2 + 1 = 32 \times 32$$

2. Compute the trainable parameters of a convolution layer with 64 filters of size 3×3 and an RGB input.

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 64 = 1792$$

3. Compare ReLU and sigmoid activation functions.

Summary: 'relu': 0.6583999991416931, 'sigmoid': 0.47189998626708984

4. Replace max pooling with average pooling and compare the results.

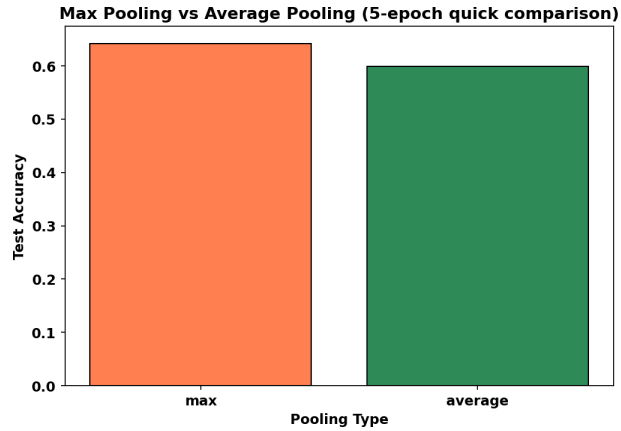


Figure 9: max pooling

5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

Filters=64: Accuracy=0.6887, Time=954.78s, Params=600650

Summary: With 16 filters, the accuracy was 0.6261, the training time was 203.74 seconds, and the model had 136,874 parameters. With 64 filters, the accuracy was 0.6887, the training time was 954.78 seconds, and the model had 600,650 parameters.

11. Results

Record

- Training Accuracy
- Testing Accuracy
- Precision
- Recall
- F1-score
- Number of Parameters

Table 1: Results

Metric	Value
Training accuracy	0.8948
Testing accuracy	0.6614
Precision	0.6585
Recall	0.6614
F1-score	0.6565
Total parameters	545,098
Total training time	1,632.28 seconds

12. Discussion

Answer the following questions:

1. Why is convolution preferred over fully connected layers for images?
Convolution neural network in this experiment used local, weight-shared filters that scans the whole image, so it requires lesser parameters than a fully connected layer while still capturing spatial patterns. Meanwhile fully connected neural network layers would have hadn to learn ever poxel position independently which is computationally expensive
2. How does stride affect the feature-map size?
A large sized (bigger square matrix like 11x11) stride moves the kernel by more pixels each step, so it skips over overlapping positions and produces a smaller feature map while a smaller stride moves the kernel with les pixels each step allowing more overlap
3. What is the role of padding?
Padding adds extra zeroes around the input border of the matrize so the kernel one can be applied to edge pixels a comparable number of times as interior pixels and also padding helps in the output preservation of the input's spatial size instead of shrinking after every convolution.
4. Why is pooling used?
Pooling downsizes feature maps to reduce the number of parameters in the spatial dimensions and to this, it also adds translation invariance to small shifts in the input, which helps control overfitting by summarizing local regions into a single representative value.
5. How do feature maps represent image characteristics?
Each feature map is a response of one learned filter across the whole image early layer maps typically highlight low level cues like edges and colour contrast, while deeper-layer maps combine these into increasingly abstract, class-relevant patterns.
6. Why do CNNs require fewer parameters than multilayer perceptrons?
CNN share the same small set of kernel weights across every spatial location and learn to share weights instead of learning a unique weight for every neuron connection as a neural network does,

so the parameter count is mostly the kernel size and filter count rather than on the full image resolution.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow documentation.
5. CIFAR-10 dataset documentation.